

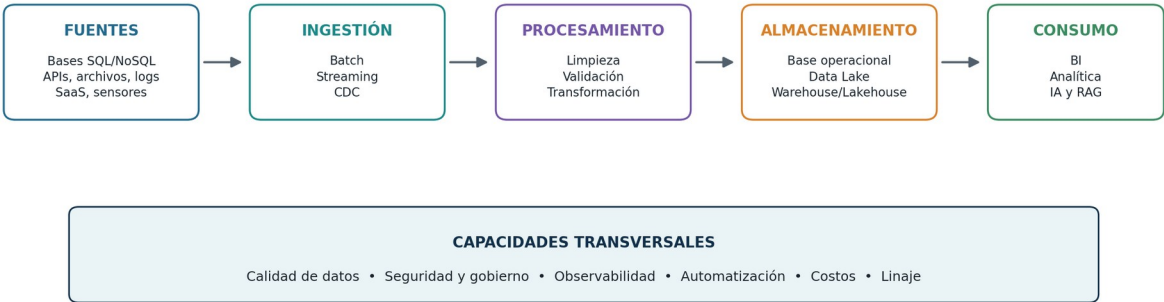
DATA ENGINEER

Guía integral del rol, funciones, tecnologías y conocimientos

Desde los fundamentos de SQL y Python hasta arquitecturas cloud, streaming, gobierno de datos e inteligencia artificial

El ciclo de vida de los datos que diseña y opera un Data Engineer

Los datos deben ser confiables, oportunos, trazables, seguros y utilizables.



Visión general del ciclo de vida de los datos

Documento explicativo y formativo

Actualizado: julio de 2026

Presentación

Esta guía explica de manera integral qué hace un Data Engineer, por qué su trabajo es esencial y qué conocimientos necesita para desarrollarse profesionalmente.

Un Data Engineer o Ingeniero de Datos diseña, construye, automatiza, mantiene y protege los sistemas que recopilan, transportan, transforman y almacenan datos. Su responsabilidad no termina cuando los datos llegan a una tabla: también debe asegurar que sean correctos, oportunos, trazables, seguros, escalables y útiles para analítica, aplicaciones e inteligencia artificial. [1][2]

IDEA CENTRAL

El Data Engineer convierte datos dispersos y difíciles de utilizar en productos de datos confiables que otras personas y sistemas pueden consumir.

Contenido

1. El rol del Data Engineer
2. El ciclo de vida de los datos
3. Pipelines, ETL y ELT
4. Almacenamiento y arquitectura
5. Modelado de datos
6. Calidad, observabilidad y operación
7. Seguridad, gobierno, metadatos y linaje
8. Tecnologías y herramientas
9. Conocimientos fundamentales
10. Diferencias con otros roles
11. Data Engineering aplicado a IA y RAG
12. Niveles profesionales y trabajo cotidiano
13. Roadmap de aprendizaje
14. Proyectos para construir experiencia
15. Glosario y bibliografía

CÓMO LEER ESTA GUÍA

Las herramientas cambian con el tiempo. Lo más importante es comprender los principios: cómo se originan los datos, cómo se mueven, cómo se transforman, cómo se almacenan, cómo se validan y cómo se operan de forma segura.

1. El rol del Data Engineer

El Ingeniero de Datos es el puente entre las fuentes de información y quienes necesitan utilizarla para tomar decisiones, automatizar procesos o construir productos digitales.

1.1. Misión principal

Su misión consiste en crear una plataforma de datos capaz de recibir información desde múltiples sistemas y entregarla en condiciones adecuadas para su uso. Las responsabilidades reconocidas por proveedores cloud incluyen diseñar sistemas de procesamiento, ingerir y transformar información, administrar almacenamiento, automatizar cargas, supervisar operaciones, optimizar rendimiento y aplicar seguridad y gobierno. [1][2][3]

- Recopilar información desde bases de datos, aplicaciones, APIs, archivos, sensores y plataformas externas.
- Diseñar pipelines repetibles para mover y transformar datos.
- Organizar el almacenamiento según el tipo de uso: transaccional, analítico, histórico o en tiempo real.
- Garantizar calidad, disponibilidad, trazabilidad y seguridad.
- Optimizar consultas, recursos de cómputo y costos.
- Proporcionar datos confiables a analistas, científicos de datos, aplicaciones, modelos de machine learning y sistemas RAG.

1.2. Resultado esperado de su trabajo

Característica	Pregunta que debe poder responder
Confiabilidad	¿Los datos representan correctamente la realidad?
Oportunidad	¿Llegan dentro del tiempo requerido?
Escalabilidad	¿El sistema soportará más volumen y usuarios?
Trazabilidad	¿Se puede conocer el origen y cada transformación?
Seguridad	¿Solo acceden las personas y servicios autorizados?
Usabilidad	¿El modelo es comprensible y útil para sus consumidores?
Eficiencia	¿La solución cumple su objetivo con un costo razonable?

1.3. Ejemplo empresarial

Una empresa puede tener ventas en una tienda web, clientes en PostgreSQL, pagos en una API externa, archivos Excel enviados por sucursales y registros de actividad en aplicaciones. El Data Engineer integra esas fuentes, valida los datos, elimina duplicados, corrige formatos, conserva el historial y publica tablas preparadas para reportes, modelos predictivos o aplicaciones.

FUENTES → INGESTIÓN → TRANSFORMACIÓN → ALMACENAMIENTO → CONSUMO

Web / PostgreSQL / APIs / Excel / Logs

↓
Datos limpios, documentados, protegidos y disponibles

1.4. Lo que no significa ser Data Engineer

- No es únicamente copiar archivos o ejecutar consultas SQL.
- No es solamente administrar una base de datos, aunque necesita conocimientos sólidos de bases.
- No es lo mismo que analizar datos y construir dashboards, aunque colabora estrechamente con quienes lo hacen.
- No es necesario dominar todas las herramientas del mercado; sí debe comprender los conceptos que permiten elegirlos correctamente.

2. El ciclo de vida de los datos

Los datos pasan por varias etapas desde que se generan hasta que producen valor. El Data Engineer diseña y mantiene ese recorrido completo.

El ciclo de vida de los datos que diseña y opera un Data Engineer

Los datos deben ser confiables, oportunos, trazables, seguros y utilizables.

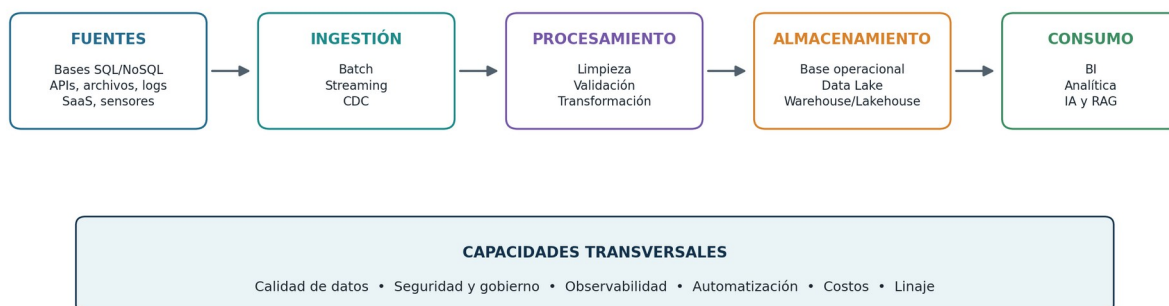


Figura 1. Ciclo de vida típico de los datos.

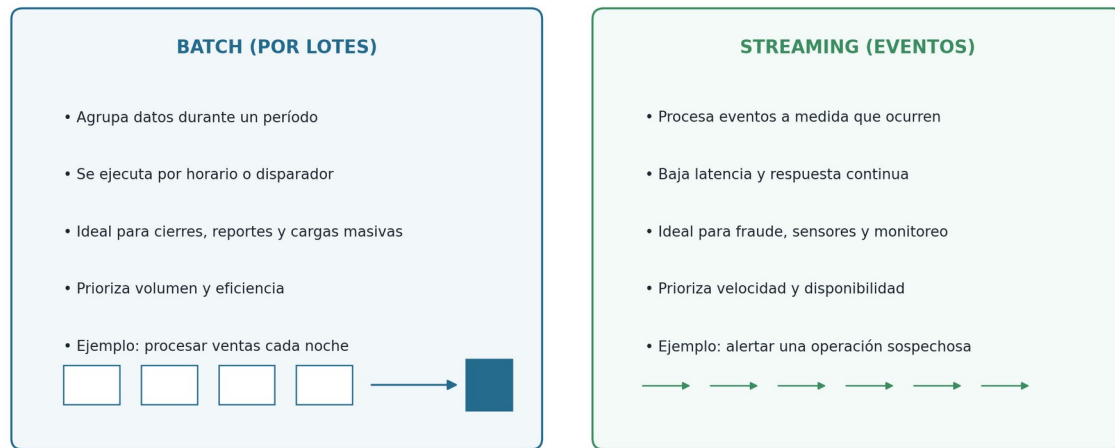
2.1. Fuentes de datos

- Bases de datos relacionales: PostgreSQL, MySQL, SQL Server u Oracle.
- Bases NoSQL: MongoDB, Cassandra, DynamoDB o Redis.
- APIs REST y GraphQL.
- Archivos CSV, Excel, JSON, XML, Parquet, Avro u ORC.
- Registros o logs de aplicaciones y servidores.
- Sistemas empresariales SaaS, CRM, ERP y plataformas de pago.
- Sensores, dispositivos IoT y eventos de usuarios.
- Documentos, imágenes, audio y contenido no estructurado.

2.2. Ingestión de datos

La ingestión incorpora datos a la plataforma. Puede realizarse por lotes, en tiempo real o mediante captura de cambios de una base de datos. La elección depende de la latencia requerida, el volumen, la complejidad y el costo.

Procesamiento batch y streaming



6. Guardar resultados en una zona de destino.
7. Ejecutar pruebas de calidad.
8. Publicar métricas y logs.
9. Reintentar o alertar si ocurre un fallo.

PROPIEDAD CLAVE: IDEMPOTENCIA

Un pipeline idempotente puede ejecutarse nuevamente sin duplicar o corromper los resultados. Es fundamental para recuperarse de fallos y reintentos.

3.2. ETL

ETL significa Extract, Transform, Load: extraer, transformar y cargar. La transformación se ejecuta antes de almacenar el resultado final. Es útil cuando el destino requiere una estructura estricta, cuando los datos sensibles deben limpiarse previamente o cuando el sistema de destino tiene capacidades limitadas. Google Cloud define ETL como un proceso de integración que extrae datos de diversas fuentes, los transforma y los carga en un sistema objetivo. [4]

EXTRACT	TRANSFORM	LOAD
API/BD/Archivo	→ limpiar, unir, validar	→ Data Warehouse

3.3. ELT

ELT significa Extract, Load, Transform. Primero se conserva la información en el entorno de almacenamiento y luego se transforma utilizando la capacidad de cómputo del Data Warehouse o Lakehouse. Este enfoque facilita mantener datos crudos, reproducir transformaciones y aprovechar plataformas cloud escalables. [5]

EXTRACT	LOAD	TRANSFORM
API/BD/Archivo	→ Data Lake/Warehouse	→ modelos SQL/dbt

3.4. Transformaciones habituales

- Convertir tipos de datos y unidades.
- Normalizar fechas, zonas horarias, monedas y códigos.
- Eliminar duplicados y resolver claves.
- Tratar valores nulos o registros incompletos.
- Combinar fuentes mediante joins.
- Aplicar reglas del negocio.
- Generar métricas, agregaciones y dimensiones.
- Anonimizar o enmascarar información sensible.
- Crear tablas de consumo con nombres y definiciones comprensibles.

3.5. Orquestación con DAG

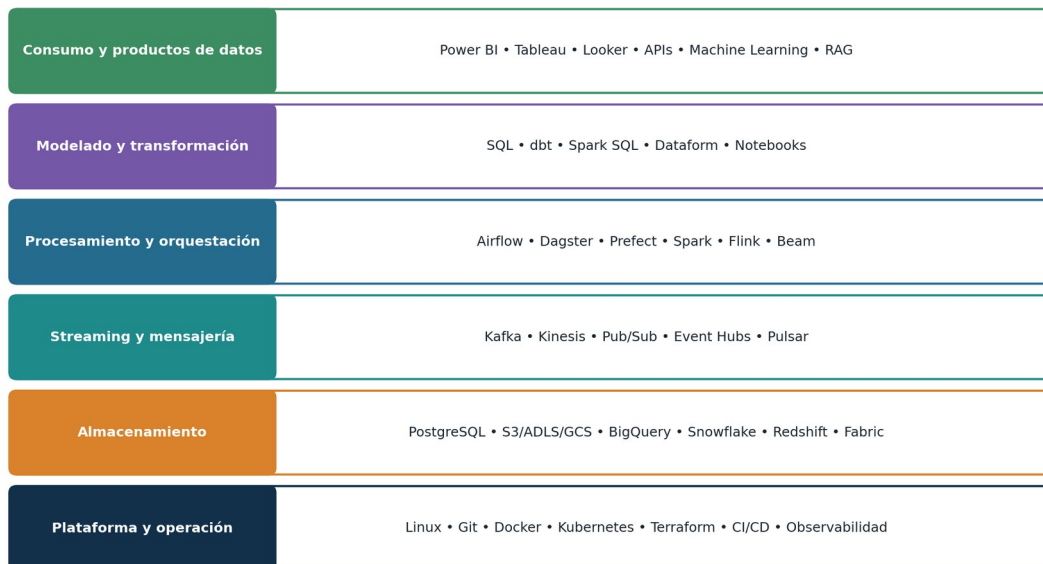
Apache Airflow representa un workflow mediante un DAG, un grafo dirigido acíclico que reúne tareas y sus dependencias. Airflow se utiliza para desarrollar, programar y supervisar workflows orientados principalmente a procesamiento batch. [6][7]

```
with DAG('ventas_diarias', schedule='0 2 * * *') as dag:
    extraer >> validar >> transformar >> cargar >> probar_calidad
```

4. Almacenamiento y arquitectura de datos

El almacenamiento no se elige únicamente por capacidad. Debe responder al patrón de uso, volumen, velocidad, formato, seguridad, latencia, costo y ciclo de vida de la información.

Arquitectura tecnológica típica de Data Engineering



La herramienta concreta puede variar; los principios de arquitectura, calidad y operación permanecen.

Figura 3. Capas y tecnologías habituales de una plataforma de datos.

4.1. Bases de datos operacionales

Los sistemas OLTP procesan transacciones cotidianas: registrar una venta, actualizar un cliente o confirmar un pago. Requieren consistencia, baja latencia y control de concurrencia. Entre las tecnologías habituales se encuentran PostgreSQL, MySQL, SQL Server, Oracle, MongoDB, DynamoDB y Cassandra.

4.2. Data Warehouse

Un Data Warehouse está orientado al análisis histórico y a consultas que agregan grandes cantidades de registros. Suele organizar datos por áreas de negocio y utilizar almacenamiento columnar. Ejemplos: BigQuery, Snowflake, Amazon Redshift, Azure Synapse y Microsoft Fabric Warehouse. [8]

4.3. Data Lake

Un Data Lake conserva grandes volúmenes de datos estructurados, semiestructurados y no estructurados. Puede almacenar archivos CSV, JSON, Parquet, imágenes, audio, logs y documentos. Ejemplos de almacenamiento de objetos son Amazon S3, Azure Data Lake Storage y Google Cloud Storage.

4.4. Lakehouse

El enfoque Lakehouse intenta combinar la flexibilidad del Data Lake con funciones de gestión propias de un Data Warehouse: tablas, evolución de esquemas, transacciones, control de versiones y mejor rendimiento analítico. Delta Lake e Apache Iceberg son formatos de tabla utilizados en este tipo de arquitectura. Apache Iceberg se define como un formato abierto de tablas para grandes conjuntos de datos analíticos. [9][10]

4.5. Cómo seleccionar el almacenamiento

Pregunta	Implicación arquitectónica
¿El sistema procesa transacciones o análisis?	OLTP para operaciones; Warehouse/Lakehouse para analítica.
¿Qué latencia necesita?	Tiempo real, casi real o procesamiento periódico.
¿Los datos tienen esquema estable?	Modelo relacional estricto o almacenamiento flexible.
¿Cuánto volumen y crecimiento se espera?	Particionamiento, escalado horizontal y almacenamiento de objetos.
¿Cómo se consulta?	Accesos por clave, joins, agregaciones, búsquedas o vectores.
¿Qué regulaciones se aplican?	Residencia, cifrado, retención, auditoría y controles de acceso.
¿Cuál es el presupuesto?	Separación de cómputo y almacenamiento, compresión y políticas de ciclo de vida.

4.6. Formatos de archivos

Formato	Fortalezas	Uso frecuente
CSV	Simple y universal; sin tipos complejos	Intercambio básico y cargas pequeñas
JSON	Flexible y jerárquico	APIs y eventos
Avro	Esquema y serialización binaria	Mensajería y evolución de esquemas
Parquet	Columnar, comprimido y eficiente	Analítica y Data Lakes
ORC	Columnar y optimizado	Ecosistema Hadoop
Delta/Iceberg/Hudi	Metadatos de tabla y operaciones avanzadas	Arquitecturas Lakehouse

Apache Parquet dispone de una especificación oficial que define cómo se estructuran y almacenan los datos en formato columnar. [11]

5. Modelado de datos

El modelado representa entidades, relaciones, reglas y formas de acceso. Un buen modelo mejora la comprensión, la consistencia y el rendimiento.

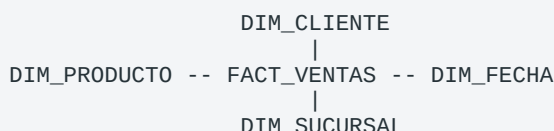
5.1. Modelado relacional

- Tablas y columnas.
- Claves primarias y foráneas.
- Relaciones uno a uno, uno a muchos y muchos a muchos.
- Restricciones de integridad.
- Normalización y desnormalización.
- Índices, vistas y transacciones.

```
CREATE TABLE ventas (
  venta_id      BIGSERIAL PRIMARY KEY,
  cliente_id    BIGINT NOT NULL REFERENCES clientes(cliente_id),
  fecha_venta   TIMESTAMP NOT NULL,
  importe       NUMERIC(14,2) CHECK (importe >= 0)
);
```

5.2. Modelado dimensional

El modelado dimensional organiza los datos para análisis. Una tabla de hechos contiene eventos y medidas; las dimensiones describen el contexto. El esquema estrella simplifica consultas y permite analizar una métrica por cliente, producto, fecha, sucursal u otras perspectivas.



- Tabla de hechos: ventas, pagos, movimientos o eventos.
- Dimensiones: cliente, producto, fecha, ubicación, canal.
- Granularidad: nivel exacto representado por cada fila.
- Medidas: importe, cantidad, duración o costo.
- Dimensiones lentamente cambiantes: conservación de cambios históricos.

5.3. Modelado orientado al acceso

En tecnologías NoSQL el diseño suele comenzar por los patrones de consulta. En lugar de normalizar todo, se puede duplicar información de manera controlada para obtener respuestas rápidas. AWS destaca que el modelado comunica propósito, alcance y diseño, y que la implementación física depende de la tecnología elegida. [12]

6. Calidad, observabilidad y operación

Una plataforma de datos valiosa no solo procesa información: demuestra que los resultados son confiables y permite detectar rápidamente cuándo algo falla.

6.1. Dimensiones de calidad

Dimensión	Significado	Ejemplo de control
Exactitud	Representa correctamente la realidad	Importes y estados coinciden con el sistema fuente
Integridad	No faltan datos necesarios	Campos obligatorios no nulos
Consistencia	No existen contradicciones	Mismo identificador y definición entre sistemas
Unicidad	No hay duplicados indebidos	Una venta no se carga dos veces
Validez	Cumple formatos y reglas	Fechas válidas e importes no negativos
Oportunidad	Llega dentro del plazo	Carga diaria finalizada antes de las 07:00
Trazabilidad	Se conoce origen y transformación	Linaje desde la fuente hasta el dashboard

6.2. Pruebas de datos

- Pruebas de esquema: columnas, tipos y obligatoriedad.
- Pruebas de unicidad y claves.
- Pruebas de relaciones entre tablas.
- Pruebas de rango y dominio.
- Comparación de recuentos entre origen y destino.
- Detección de anomalías de volumen o distribución.
- Pruebas de regresión de transformaciones.

```
-- Registros duplicados
SELECT venta_id, COUNT(*)
FROM ventas
GROUP BY venta_id
HAVING COUNT(*) > 1;

-- Valores inválidos
SELECT * FROM ventas
WHERE importe < 0 OR fecha_venta IS NULL;
```

6.3. Observabilidad

La observabilidad permite comprender el estado interno del sistema mediante métricas, logs, trazas, alertas y contexto del linaje. Un pipeline debe informar cuánto procesó, cuánto demoró, qué etapa falló y qué datos fueron afectados.

Indicador	Qué informa
Duración	Tiempo total y por tarea
Freshness	Antigüedad del último dato disponible
Volumen	Registros o bytes procesados
Errores	Cantidad y tipo de fallos
Calidad	Pruebas superadas y fallidas
Recursos	CPU, memoria, red y almacenamiento
Costo	Consumo por pipeline, equipo o producto de datos
SLA/SLO	Cumplimiento del nivel de servicio esperado

6.4. Recuperación y soporte

- Reintentos con espera progresiva.
- Dead-letter queues para eventos que no pueden procesarse.
- Checkpoints y recuperación de estado.
- Backfill para reprocesar períodos históricos.
- Copias de seguridad y restauración.
- Runbooks con pasos de diagnóstico y solución.
- Gestión de incidentes y análisis de causa raíz.

7. Seguridad, gobierno, metadatos y linaje

Los datos pueden ser sensibles y estar sujetos a regulaciones. La seguridad debe diseñarse desde el inicio y no agregarse al final.

7.1. Seguridad

- Autenticación de usuarios y servicios.
- Autorización basada en roles y principio de mínimo privilegio.
- Cifrado en tránsito y en reposo.
- Gestión segura de contraseñas, claves y secretos.
- Enmascaramiento, tokenización o anonimización.
- Controles por fila, columna o dominio.
- Auditoría de accesos y cambios.
- Segmentación de redes y endpoints privados.

El dominio de seguridad y gobierno de la certificación AWS Data Engineer incluye autenticación, autorización, cifrado, enmascaramiento, privacidad, auditoría y controles de gobierno. [3]

7.2. Gobierno de datos

El gobierno establece quién es responsable de los datos, qué definiciones se utilizan, quién puede acceder, cuánto tiempo se conservan y qué controles deben aplicarse. Incluye políticas, procesos, roles, estándares y evidencias de cumplimiento.

- Data Owner: responsable de decisiones y uso del dato.
- Data Steward: cuida definiciones, calidad y reglas.
- Data Custodian: administra técnicamente la plataforma.
- Clasificación: público, interno, confidencial o restringido.
- Retención: tiempo de conservación y eliminación.
- Catálogo: inventario buscable de conjuntos de datos.

7.3. Metadatos y catálogo

Los metadatos describen tablas, columnas, propietarios, significado, sensibilidad, actualización y uso. Un catálogo permite descubrir activos de datos y comprender si son apropiados para una necesidad concreta.

7.4. Linaje

```
Sistema de ventas
↓
ventas_raw
↓ limpieza y conversión de moneda
ventas_curadas
↓ modelo dimensional
fact_ventas
↓
Dashboard de facturación
```

POR QUÉ IMPORTA EL LINAJE

Permite saber de dónde proviene una cifra, evaluar el impacto de un cambio, investigar errores y demostrar cumplimiento.

8. Tecnologías y herramientas

El ecosistema es amplio. La siguiente clasificación ayuda a comprender para qué sirve cada grupo de herramientas.

8.1. Lenguajes

Lenguaje	Uso principal	Prioridad inicial
SQL	Consulta, transformación, modelado y validación	Muy alta
Python	Pipelines, APIs, automatización, pruebas y procesamiento	Muy alta
Bash/Shell	Automatización y operación en Linux	Alta
YAML	Configuración de CI/CD, contenedores y orquestación	Media
Scala	Spark y procesamiento distribuido	Según el entorno
Java	Kafka, Spark y ecosistemas empresariales	Según el entorno
HCL	Infraestructura como código con Terraform	Media
PowerShell	Automatización en entornos Microsoft	Según el entorno

8.2. Bases de datos

Categoría	Tecnologías	Conceptos que debe dominar
Relacionales	PostgreSQL, MySQL, SQL Server, Oracle	Joins, transacciones, índices, planes de ejecución, bloqueos
Documentos	MongoDB, Couchbase	Documentos, índices, consistencia y patrones de acceso
Clave-valor	Redis, DynamoDB	Partición, TTL, acceso por clave
Columnas anchas	Cassandra	Distribución, particiones y consistencia
Búsqueda	Elasticsearch, OpenSearch	Índices invertidos, texto y agregaciones
Vectoriales	pgvector, Milvus, Pinecone, Weaviate	Embeddings, similitud y filtros de metadatos

8.3. Procesamiento

- pandas y Polars para procesamiento local.
- Apache Spark y PySpark para procesamiento distribuido.
- Apache Flink para streaming con estado.
- Apache Beam como modelo unificado de procesamiento.
- Servicios administrados: AWS Glue, Google Dataflow, Databricks y Microsoft Fabric.

Apache Spark ofrece APIs para Python, Scala y Java y módulos para datos estructurados. La documentación de PySpark describe Spark SQL como el módulo para trabajar con datos estructurados. [13][14]

8.4. Streaming y mensajería

- Apache Kafka y Kafka Connect.
- Amazon Kinesis.
- Google Cloud Pub/Sub.

- Azure Event Hubs.
- Apache Pulsar.
- RabbitMQ para colas y mensajería tradicional.

Apache Kafka se define como una plataforma de event streaming. Puede publicar, almacenar y procesar flujos de eventos, y suele utilizarse como columna vertebral de arquitecturas orientadas a eventos. [15]

8.5. Orquestación y transformación analítica

- Apache Airflow, Dagster, Prefect y Luigi para orquestación.
- dbt y Dataform para transformaciones analíticas versionadas.
- Azure Data Factory, Fabric Data Factory, AWS Step Functions y Cloud Composer como servicios administrados.

8.6. Cloud computing

AWS	Azure / Microsoft Fabric	Google Cloud
S3	ADLS / OneLake	Cloud Storage
Redshift	Fabric Warehouse / Synapse	BigQuery
Glue / EMR	Fabric / Databricks	Dataflow / Dataproc
Kinesis / MSK	Event Hubs	Pub/Sub
Step Functions / MWAA	Data Factory	Cloud Composer
CloudWatch / IAM	Azure Monitor / Entra ID	Cloud Monitoring / IAM

Microsoft describe al Data Engineer de Fabric como responsable de ingerir, transformar, supervisar y optimizar soluciones de datos mediante componentes de la plataforma. [16]

8.7. DevOps y DataOps

- Git, GitHub, GitLab o Bitbucket para control de versiones.
- GitHub Actions, GitLab CI/CD, Jenkins o Azure DevOps para integración y despliegue.
- Docker para empaquetar aplicaciones y dependencias en contenedores. [17]
- Kubernetes para administrar cargas de trabajo contenerizadas mediante configuración declarativa y automatización. [18]
- Terraform para definir recursos cloud y on-premises como código versionable y reutilizable. [19]
- Prometheus, Grafana y servicios cloud para monitoreo.

DATAOPS

DataOps aplica prácticas de ingeniería de software y operaciones al ciclo de vida de los datos: versionado, pruebas, automatización, despliegues controlados, observabilidad y mejora continua.

9. Conocimientos fundamentales

La profesión combina programación, bases de datos, sistemas distribuidos, operaciones, seguridad y comprensión del negocio.

9.1. SQL avanzado

- SELECT, INSERT, UPDATE y DELETE.
- JOIN, subconsultas y CTE.
- Funciones de ventana.
- Agregaciones y agrupamientos.
- Vistas, procedimientos y funciones.
- Índices y planes de ejecución.
- Transacciones, aislamiento y bloqueos.
- Particionamiento y optimización.

```
SELECT
  cliente_id,
  fecha_venta,
  importe,
  SUM(importe) OVER (
    PARTITION BY cliente_id
    ORDER BY fecha_venta
  ) AS acumulado_cliente
FROM ventas;
```

9.2. Python

- Funciones, módulos, clases y excepciones.
- Entornos virtuales, paquetes y dependencias.
- Lectura y escritura de archivos.
- Consumo de APIs.
- Conexión a bases de datos.
- Logging, configuración y gestión de secretos.
- Pruebas con pytest.
- Librerías: pandas, PySpark, SQLAlchemy, requests, Pydantic, pyarrow y SDK cloud.

9.3. Bases de datos y sistemas distribuidos

- OLTP frente a OLAP.
- Propiedades ACID y consistencia.
- Replicación, particionamiento y sharding.
- Escalado vertical y horizontal.
- Tolerancia a fallos.
- Paralelismo, serialización y shuffle.
- Throughput, latencia y backpressure.
- Semánticas de entrega: al menos una vez, como máximo una vez y exactamente una vez.

9.4. Ingeniería de software

- Separación de responsabilidades.
- Manejo explícito de errores.
- Configuración por ambiente.
- Pruebas unitarias, de integración y end-to-end.
- Revisión de código y ramas de Git.
- APIs y contratos de datos.
- CI/CD y despliegues reproducibles.
- Documentación técnica y runbooks.

9.5. Linux y redes

- Sistema de archivos, permisos, usuarios y procesos.
- Variables de entorno, servicios y cron.
- SSH y transferencia de archivos.
- Puertos, DNS, TCP/IP, HTTP y HTTPS.
- Certificados, firewalls y redes privadas.
- Herramientas de logs y diagnóstico.

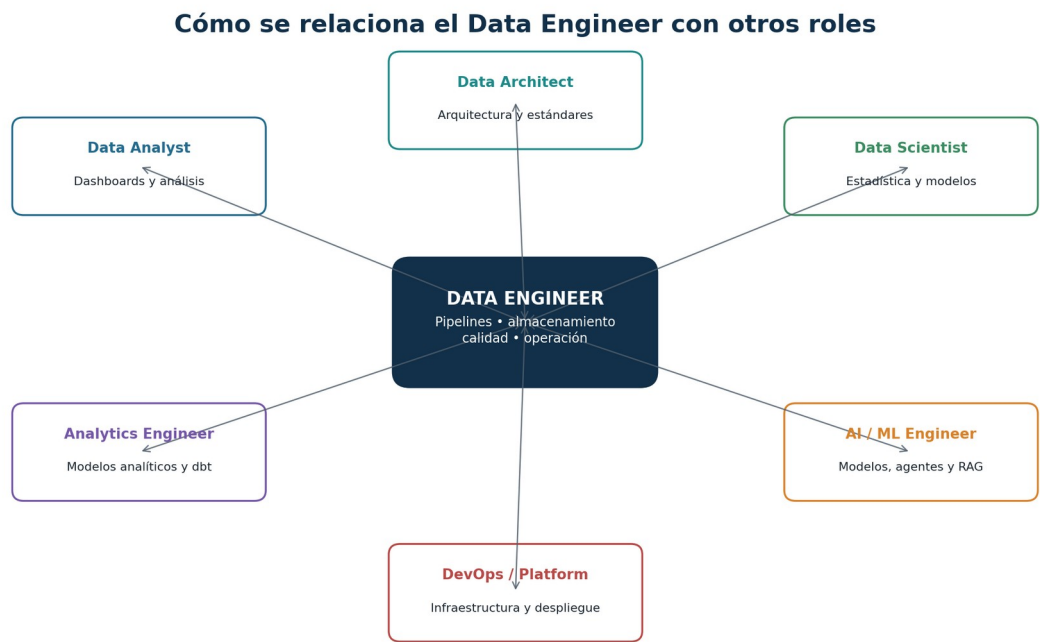
9.6. Matemática, estadística y negocio

- Estadística descriptiva, percentiles, dispersión y valores atípicos.
- Probabilidad básica y muestreo.
- Lógica, conjuntos y funciones.
- Interpretación de métricas.
- Definiciones del negocio: qué es una venta, un cliente activo, una devolución o un cierre.
- Comunicación con usuarios técnicos y no técnicos.

RIESGO HABITUAL

Un pipeline puede ser técnicamente correcto y, sin embargo, producir un indicador equivocado si la regla del negocio fue interpretada de manera incorrecta.

10. Diferencias con otros roles



En equipos pequeños, una persona puede cubrir varias funciones; en organizaciones grandes, las responsabilidades se especializan.

Figura 4. El Data Engineer dentro del ecosistema de datos, IA y plataforma.

Rol	Responsabilidad principal	Entrega habitual
Data Engineer	Infraestructura, pipelines, almacenamiento y calidad	Tablas, streams, APIs y productos de datos
Data Analyst	Analiza información y comunica resultados	Dashboards e informes

Rol	Responsabilidad principal	Entrega habitual
Analytics Engineer	Modela la capa analítica	Modelos SQL/dbt y métricas
Data Scientist	Analiza patrones y crea modelos	Experimentos y modelos predictivos
ML Engineer	Industrializa modelos de machine learning	Servicios de inferencia y pipelines ML
AI Engineer	Construye aplicaciones con modelos generativos	Agentes, RAG, prompts y evaluaciones
DBA	Administra motores de bases de datos	Disponibilidad, respaldo y rendimiento
Data Architect	Define arquitectura y estándares	Diseños, principios y modelos de referencia
DevOps/Platform	Automatiza infraestructura y despliegues	Plataformas, CI/CD y observabilidad
Data Governance	Define políticas y responsabilidades	Catálogo, clasificación y controles

10.1. Data Engineer y DevOps

El Data Engineer se concentra en datos, pipelines, almacenamiento y calidad. El DevOps o Platform Engineer se concentra en infraestructura, despliegue, confiabilidad y operación de plataformas. Ambos pueden utilizar Linux, Git, Docker, Kubernetes, Terraform, cloud, CI/CD y monitoreo. La intersección suele denominarse DataOps.

10.2. Data Engineer y Analytics Engineer

El Analytics Engineer se ubica entre la ingeniería y el análisis. Trabaja principalmente en la capa de transformación y modelado analítico, con herramientas como SQL y dbt. El Data Engineer suele responsabilizarse además por ingestión, plataformas, procesamiento distribuido, seguridad y operación.

11. Data Engineering aplicado a inteligencia artificial y RAG

Los sistemas de IA dependen de datos accesibles, actualizados y de calidad. Un modelo potente no compensa una base documental desordenada o sin control de permisos.

11.1. Aportes del Data Engineer

- Conectar repositorios, bases de datos, APIs y sistemas documentales.
- Extraer texto y metadatos.
- Limpiar, normalizar y eliminar duplicados.
- Controlar versiones y vigencia de documentos.
- Aplicar permisos y clasificación de sensibilidad.
- Automatizar cargas incrementales.
- Preparar datos para segmentación e indexación.
- Mantener linaje entre fuente, fragmento, embedding y respuesta.
- Medir cobertura, freshness y fallos de actualización.

11.2. Distribución de responsabilidades en RAG

Rol	Responsabilidades en un sistema RAG
Data Engineer	Fuentes, ingestión, limpieza, metadatos, actualización,

Rol	Responsabilidades en un sistema RAG
	calidad y almacenamiento.
AI Engineer	Chunking, embeddings, retrieval, prompts, evaluación y guardrails.
ML/MLOps Engineer	Despliegue, versiones, escalado, observabilidad y ciclo de vida del modelo.
Security/Governance	Permisos, privacidad, auditoría, retención y cumplimiento.
Domain Expert	Valida contenido, reglas y calidad de las respuestas.

11.3. Pipeline documental de referencia

```

Documentos / Bases / APIs
↓
Extracción y normalización
↓
Deduplicación + metadatos + permisos
↓
Chunking y embeddings
↓
Índice vectorial + almacenamiento documental
↓
Retrieval → modelo generativo → respuesta con fuentes

```

PRINCIPIO DE SEGURIDAD

El sistema de recuperación no debe devolver información que el usuario original no estaba autorizado a consultar.

12. Niveles profesionales y trabajo cotidiano

12.1. Junior

- SQL y Python básicos.
- Git, Linux y PostgreSQL.
- APIs, CSV y JSON.
- ETL simple y Docker básico.
- Corrección de pipelines existentes y tareas bien definidas.

12.2. Semi Senior

- Modelado dimensional y optimización SQL.
- Airflow, dbt y cloud intermedio.
- Spark y streaming básico.
- Pruebas, CI/CD y observabilidad.
- Diseño de pipelines completos y resolución de incidentes.

12.3. Senior

- Arquitectura y sistemas distribuidos.
- Seguridad, gobierno y costos.
- Alta disponibilidad y recuperación.

- Definición de estándares.
- Liderazgo técnico, evaluación de tecnologías y mentoría.

12.4. Principal Data Engineer o Data Architect

- Arquitectura empresarial y estrategia de plataforma.
- Integración entre dominios y productos de datos.
- Estándares de interoperabilidad, seguridad y gobierno.
- Plan de evolución tecnológica y gestión de riesgos.

12.5. Un día de trabajo posible

Horario	Actividad
08:00	Revisar pipelines nocturnos, alertas y frescura de datos.
08:45	Investigar un fallo y ejecutar un reproceso controlado.
10:00	Reunión con analistas y negocio para definir una nueva fuente.
11:00	Crear transformaciones SQL y pruebas de calidad.
13:30	Revisar código de un compañero.
14:30	Optimizar una consulta o un job de Spark.
16:00	Desplegar cambios mediante CI/CD.
17:00	Actualizar documentación, métricas y runbook.

13. Roadmap de aprendizaje

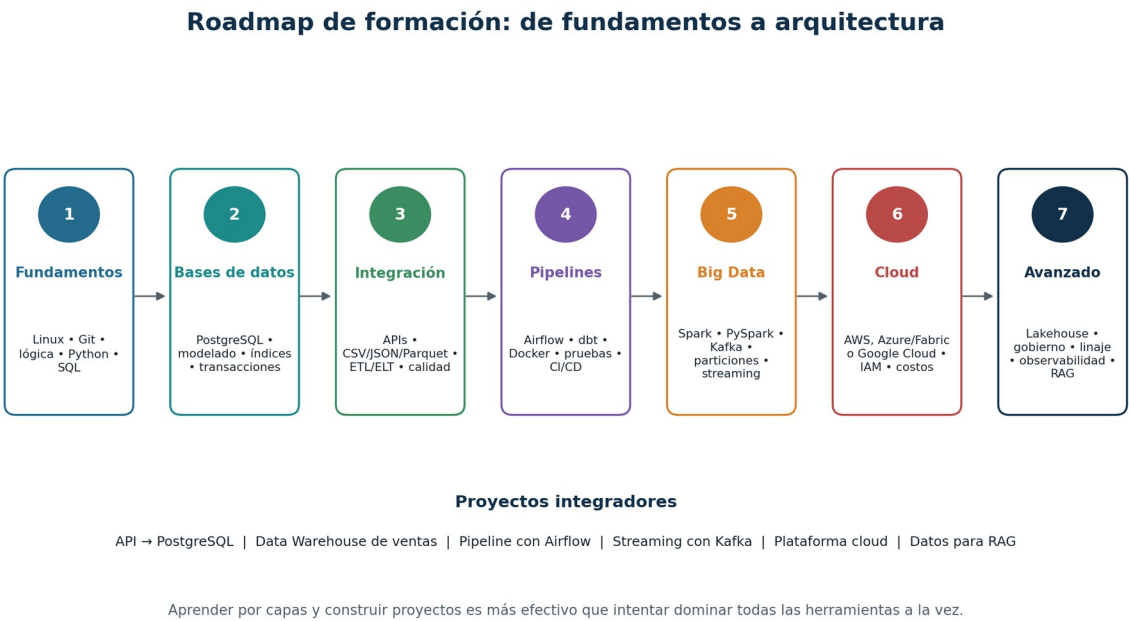


Figura 5. Ruta progresiva de formación.

13.1. Etapa 1: fundamentos

1. Linux y terminal.
2. Git y GitHub.
3. Lógica de programación.
4. Python.
5. SQL.
6. PostgreSQL.
7. HTTP y redes básicas.

13.2. Etapa 2: bases de datos

1. Modelado relacional y normalización.
2. Índices y planes de ejecución.
3. Transacciones, concurrencia y bloqueos.
4. Backups y restauración.
5. Introducción a NoSQL.

13.3. Etapa 3: integración

1. APIs REST.
2. CSV, JSON, Parquet y esquemas.
3. pandas o Polars.
4. ETL y ELT.
5. Validación, logging y manejo de errores.

13.4. Etapa 4: pipelines profesionales

1. Apache Airflow.
2. dbt.
3. Docker.
4. Pruebas automáticas.
5. CI/CD.
6. Monitoreo y documentación.

13.5. Etapa 5: Big Data y streaming

1. Sistemas distribuidos.
2. Apache Spark y PySpark.
3. Particionamiento y Parquet.
4. Apache Kafka.
5. Procesamiento con estado y tolerancia a fallos.

13.6. Etapa 6: cloud

Conviene elegir inicialmente una plataforma: AWS, Azure/Microsoft Fabric o Google Cloud. Luego aprender almacenamiento, identidad y permisos, redes, cómputo, bases administradas, Data Warehouse, monitoreo, costos e infraestructura como código.

13.7. Etapa 7: avanzado

1. Lakehouse y formatos de tabla.
2. Catálogo, gobierno y linaje.
3. Observabilidad y contratos de datos.
4. Arquitectura orientada a eventos.
5. Data Mesh y productos de datos.
6. Integración con IA, embeddings y RAG.

13.8. Stack recomendado para comenzar

Área	Herramienta inicial
Sistema operativo	Linux
Lenguaje	Python
Consultas	SQL
Base de datos	PostgreSQL
Procesamiento local	pandas o Polars
Orquestación	Apache Airflow
Transformación analítica	dbt
Contenedores	Docker
Versionado	Git y GitHub
Almacenamiento tipo S3 local	MinIO
Visualización	Power BI
Etapa posterior	Cloud, Spark y Kafka

14. Proyectos para construir experiencia

Los proyectos deben demostrar no solo que el pipeline funciona, sino que puede probarse, observarse, desplegarse y recuperarse.

Proyecto 1. Pipeline desde una API

- Consumir una API pública.
- Guardar una copia cruda de la respuesta.
- Validar con Pydantic o reglas equivalentes.
- Transformar con Python.
- Cargar en PostgreSQL.
- Registrar logs y métricas.
- Programar la ejecución.

Proyecto 2. Data Warehouse de ventas

- Crear fuentes de clientes, productos, ventas y sucursales.
- Diseñar tabla de hechos y dimensiones.

- Transformar con dbt.
- Crear pruebas de integridad y unicidad.
- Publicar un dashboard en Power BI.

Proyecto 3. Orquestación con Airflow

- Construir un DAG con dependencias.
- Implementar reintentos y alertas.
- Agregar backfill.
- Separar ambientes.
- Incluir pruebas y despliegue con GitHub Actions.

Proyecto 4. Streaming

- Generar eventos simulados.
- Publicar en Kafka.
- Procesar con Spark Structured Streaming o Flink.
- Guardar resultados agregados.
- Crear un tablero de monitoreo.

Proyecto 5. Plataforma cloud

- Crear almacenamiento de objetos.
- Configurar roles y permisos.
- Implementar un pipeline administrado.
- Publicar datos en un Warehouse.
- Definir infraestructura con Terraform.
- Medir costo y rendimiento.

Proyecto 6. Datos para RAG

- Importar documentos de varias fuentes.
- Extraer texto y metadatos.
- Deduplicar y controlar versiones.
- Aplicar clasificación y permisos.
- Automatizar actualizaciones incrementales.
- Preparar la indexación vectorial y evaluar cobertura.

14.1. Criterios para un portafolio sólido

Criterio	Evidencia
Reproducibilidad	README, Docker Compose y datos de ejemplo.
Calidad	Pruebas automáticas y resultados visibles.
Observabilidad	Logs, métricas, dashboard y alertas.
Seguridad	Secretos fuera del código y permisos mínimos.
Diseño	Diagrama de arquitectura y decisiones documentadas.
Operación	Reintentos, backfill y procedimiento de recuperación.
Código	Estructura clara, Git y revisión automática.

15. Glosario

Término	Definición
API	Interfaz que permite a sistemas intercambiar información.
Batch	Procesamiento de datos agrupados en intervalos.
CDC	Captura de cambios producidos en una base de datos.
DAG	Grafo dirigido acíclico que representa tareas y dependencias.
Data Lake	Repositorio flexible para datos de diversos formatos.
Data Warehouse	Almacén optimizado para análisis.
Lakehouse	Arquitectura que combina Lake y Warehouse.
ETL	Extraer, transformar y cargar.
ELT	Extraer, cargar y transformar.
Idempotencia	Propiedad que permite repetir una operación sin alterar indebidamente el resultado.
Linaje	Registro del recorrido y transformaciones del dato.
Metadatos	Información que describe otros datos.
Orquestación	Coordinación y programación de tareas.
Particionamiento	División de datos para mejorar distribución y acceso.
SLA	Compromiso formal de nivel de servicio.
SLO	Objetivo medible de confiabilidad o rendimiento.
Streaming	Procesamiento continuo de eventos.
Throughput	Cantidad de datos procesados por unidad de tiempo.

Conclusión

Un Data Engineer completo combina capacidades de programación, bases de datos, integración, cloud, DevOps y conocimiento del negocio. Su valor principal consiste en construir sistemas que permitan confiar en los datos y utilizarlos de forma sostenible.

```

SQL + Python + Bases de datos + ETL/ELT
      ↓
Pipelines + Calidad + Orquestación + Cloud
      ↓
Spark + Kafka + Seguridad + Gobierno + Arquitectura
      ↓
Productos de datos para analítica, aplicaciones e IA
  
```

RECOMENDACIÓN FINAL

Comenzar con SQL, Python, PostgreSQL, Linux y un proyecto ETL completo. Después incorporar Airflow, dbt, Docker, cloud, Spark y Kafka. La profundidad en los fundamentos es más importante que la cantidad de herramientas conocidas.

Bibliografía y documentación oficial

Fuentes consultadas y verificadas para la elaboración de esta guía. Las páginas corresponden a documentación oficial de proveedores y proyectos.

- [1] Google Cloud - Professional Data Engineer Certification. <https://cloud.google.com/learn/certification/data-engineer>
- [2] AWS - Certified Data Engineer Associate Exam Guide. <https://docs.aws.amazon.com/aws-certification/latest/data-engineer-associate-01.html>
- [3] AWS - Data Security and Governance Domain. <https://docs.aws.amazon.com/aws-certification/latest/data-engineer-associate-01/data-engineer-associate-01-domain4.html>
- [4] Google Cloud - What is ETL?. <https://cloud.google.com/learn/what-is-etl>
- [5] Google Cloud - What is ELT?. <https://cloud.google.com/discover/what-is-elt>
- [6] Apache Airflow - What is Airflow?. <https://airflow.apache.org/docs/apache-airflow/stable/index.html>
- [7] Apache Airflow - DAGs. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>
- [8] Google Cloud - What is a Data Warehouse?. <https://cloud.google.com/learn/what-is-a-data-warehouse>
- [9] Delta Lake - Documentation. <https://docs.delta.io/>
- [10] Apache Iceberg - Documentation. <https://iceberg.apache.org/docs/latest/>
- [11] Apache Parquet - Documentation. <https://parquet.apache.org/docs/>
- [12] AWS - What is Data Modeling?. <https://aws.amazon.com/what-is/data-modeling/>
- [13] Apache Spark - Documentation. <https://spark.apache.org/documentation>
- [14] Apache Spark - PySpark Overview. <https://spark.apache.org/docs/latest/api/python/index.html>
- [15] Apache Kafka - Documentation. <https://kafka.apache.org/documentation/>
- [16] Microsoft Learn - Fabric Data Engineer Associate. <https://learn.microsoft.com/en-us/credentials/certifications/fabric-data-engineer-associate/>
- [17] Docker Docs - Docker Overview. <https://docs.docker.com/get-started/docker-overview/>
- [18] Kubernetes Documentation - Concepts. <https://kubernetes.io/docs/concepts/>
- [19] HashiCorp - What is Terraform?. <https://developer.hashicorp.com/terraform/intro>

Fin del documento